

## AWS VPC Overview



### Notes

#### What is VPC?

- **VPC (Virtual Private Cloud)** is a logically isolated section of the AWS cloud where you can launch AWS resources in a virtual network you define.
- You have complete control over your virtual networking environment, including selection of your own IP address range, creation of subnets, and configuration of route tables and network gateways.

---

#### Why is VPC Used?

- To **isolate and secure** resources.
- Allows control over:
  - **Inbound and outbound traffic** using security groups and network ACLs.
  - **Subnets** (public/private).
  - **Custom IP ranges** using CIDR blocks.
  - **Route tables and internet gateways.**
- Enables **hybrid cloud** setups using VPN or AWS Direct Connect.
- Provides the ability to **segment applications** by environment (e.g., dev, test, prod).

## Understanding CIDR IPv4

### What is CIDR?

- **CIDR (Classless Inter-Domain Routing)** notation is used to specify IP address ranges.
- Format: `IP_address/Prefix_Length`  
Example: `10.0.0.0/16`

### Components of CIDR

- **Base IP:** Starting point of the IP address range (e.g., `10.0.0.0`)
- **Subnet Mask / Prefix Length:**
  - Determines how many IP addresses are available.
  - Written as `/n` where `n` is the number of bits in the network portion.

### Example:

- `10.0.0.0/16`:
  - Base IP: `10.0.0.0`
  - Prefix Length: `16` → Subnet mask = `255.255.0.0`
  - Total IPs:  $2^{(32-16)} = 65,536$  IP addresses

### Range of CIDRs in AWS

- AWS supports the following CIDR ranges for VPCs:
  - **/16 to /28** for VPC subnetting
  - Example Ranges:
    - `10.0.0.0/16`

- 172.16.0.0/12
- 192.168.0.0/16

- The total number of subnets you can create depends on how you further divide the CIDR block.

## Public IP vs Private IP

| Feature       | Public IP                                                  | Private IP                                   |
|---------------|------------------------------------------------------------|----------------------------------------------|
| Scope         | Globally unique                                            | Unique within VPC                            |
| Accessibility | Accessible from the internet                               | Only accessible within VPC/private network   |
| Assignment    | By AWS (dynamically or elastic IP)                         | Assigned automatically by AWS or manually    |
| Use Case      | For resources that need internet access (e.g., Web Server) | For internal communication (e.g., DB server) |

## What is a Route Table in VPC?

A **Route Table** in AWS VPC is a set of rules, called **routes**, that are used to determine where network traffic is directed.

### Key Concepts:

- Every **subnet** in a VPC must be associated with a route table.
- **Default Route Table:** When you create a VPC, AWS creates a **main route table** by default.
- You can create **custom route tables** and associate them with one or more subnets.
- **Routing is determined based on destination IP ranges (CIDR blocks).**

**Route Table Components:**

| Component | Description |
|-----------|-------------|
|-----------|-------------|

**Destination** The range of IP addresses (CIDR) for which the route applies.

**Target** Where to send the traffic (e.g., local, internet gateway, NAT gateway, VPC peering connection, etc.)

**Example Routes:**

| Destination    | Target              | Description                                |
|----------------|---------------------|--------------------------------------------|
| 10.0.0.0/16    | local               | Default route within the VPC               |
| 0.0.0.0/0      | igw-xxxxxxx         | Route to the Internet Gateway              |
| 0.0.0.0/0      | nat-gw-xxxxx<br>xxx | Route to NAT Gateway (for private subnets) |
| 192.168.1.0/24 | pcx-xxxxxxx         | Route to a peered VPC via VPC peering      |

**Types of Subnets based on Routing:**

| Subnet Type            | Route to Internet Gateway? | Route Table Example                             |
|------------------------|----------------------------|-------------------------------------------------|
| <b>Public Subnet</b>   | Yes (0.0.0.0/0 → IGW)      | Allows Internet access                          |
| <b>Private Subnet</b>  | No direct IGW route        | Uses NAT Gateway if outbound Internet is needed |
| <b>Isolated Subnet</b> | No IGW, no NAT             | No outbound Internet access                     |

**Max 5 CIDRs in a VPC**

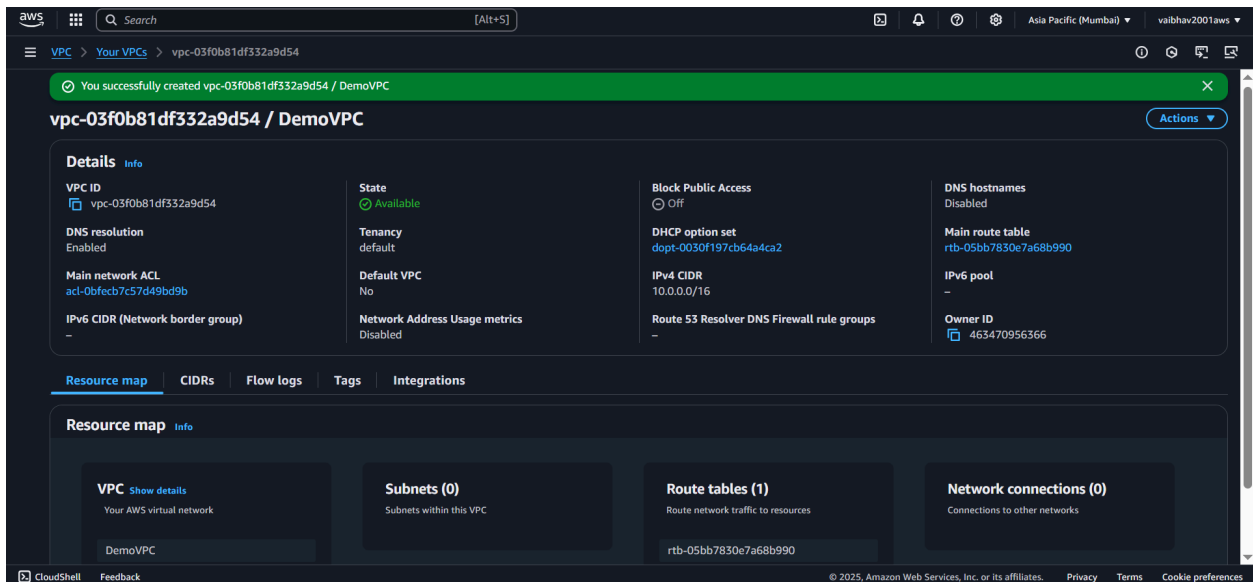
- A single VPC can have up to **5 IPv4 CIDR blocks** (including the primary one).
- This allows you to **expand your IP address range** or **bring non-overlapping IPs** into the same VPC.

**Important Notes:**

- CIDR blocks must not **overlap**.
- Each CIDR must be between **/16 and /28**.
- You can **add secondary CIDR blocks** using the **VPC dashboard or CLI**.

Example CIDR blocks in one VPC:

- 10.0.0.0/16
- 10.1.0.0/16
- 192.168.0.0/16
- 172.16.0.0/16
- 172.31.0.0/16

**Step 1 :- Create a VPC named DemoVPC with CIDR 10.0.0.0/16.****What is a Subnet in VPC?**

- A **Subnet (sub-network)** is a range of IP addresses within a VPC.
- You divide your VPC into subnets to isolate and organize resources.
- Subnets can be:
  - **Public** – if associated with a route to the Internet Gateway.
  - **Private** – if they don't have a route to the Internet.

**Key Points:**

- Subnets must reside **within a single Availability Zone**.
- Each subnet is assigned a portion of the VPC's CIDR block.
- You can control access at the **subnet level using network ACLs** and at the **resource level using security groups**.

## Why Are Subnets Important?

- Enable **segregation of resources** (e.g., web tier, app tier, DB tier).
- Facilitate **high availability** by placing subnets in multiple Availability Zones.
- Required for **routing, security, and scaling** of AWS services.

---

## AWS Reserves 5 IP Addresses in Every Subnet

For each subnet, **AWS reserves the first 4 and the last 1 IP address** in the subnet range. These IPs are **not available for use** by your EC2 instances or other services.

### Reserved IPs Explanation:

Let's assume a subnet `10.0.1.0/24`

It has 256 total IP addresses (from `10.0.1.0` to `10.0.1.255`)

| IP Address              | Reserved For                                       |
|-------------------------|----------------------------------------------------|
| <code>10.0.1.0</code>   | Network address                                    |
| <code>10.0.1.1</code>   | Reserved by AWS for VPC router                     |
| <code>10.0.1.2</code>   | Reserved by AWS for DNS (AmazonProvidedDNS)        |
| <code>10.0.1.3</code>   | Reserved by AWS for future use                     |
| <code>10.0.1.255</code> | Network broadcast address (though not used in AWS) |

**Effective usable IPs** in `10.0.1.0/24`:

`10.0.1.4` to `10.0.1.254` → **251 usable addresses**

### Subnet Sizing Tip for the Exam:

- Always consider that **5 IPs per subnet are unusable**.
- When planning subnet sizing, subtract 5 from the total for accurate capacity.

### CIDR Subnet Sizes and Usable IPs:

| CIDR Block       | Total IPs | Usable IPs |
|------------------|-----------|------------|
| <code>/28</code> | 16        | 11         |
| <code>/27</code> | 32        | 27         |
| <code>/26</code> | 64        | 59         |
| <code>/24</code> | 256       | 251        |
| <code>/16</code> | 65,536    | 65,531     |



## Step 2 :Create 4 Subnets in DemoVPC 2 public and 2 private and enable Auto-Assign Public IP

| Name             | Subnet ID                | State     | VPC                            | Block Public... | IPv4 CIDR    |
|------------------|--------------------------|-----------|--------------------------------|-----------------|--------------|
| Private Subnet b | subnet-06d9db2e220aabdfc | Available | vpc-03f0b81df332a9d54   Dem... | Off             | 10.0.32.0/20 |
| Public Subnet b  | subnet-015d5b45f88045acc | Available | vpc-03f0b81df332a9d54   Dem... | Off             | 10.0.1.0/24  |
| Public Subnet a  | subnet-0bd017b11912b3198 | Available | vpc-03f0b81df332a9d54   Dem... | Off             | 10.0.0.0/24  |
| Private Subnet a | subnet-067e47cc6547a72e7 | Available | vpc-03f0b81df332a9d54   Dem... | Off             | 10.0.16.0/20 |

## What is an Internet Gateway (IGW)?

- An **Internet Gateway (IGW)** is a **VPC component** that allows communication between resources in your VPC and the **internet**.
- It enables instances with **public IPs or Elastic IPs** to send and receive traffic from the internet.

### Key Features:

- **Horizontally scaled, redundant, and highly available.**
- Only **one IGW** can be attached to a VPC at a time.
- IGW performs **1:1 NAT** for instances with public IPs.

## Why is Internet Gateway Used with Route Table?

- IGW alone doesn't provide internet access.
- You must **explicitly create a route** in the **subnet's route table** pointing to the IGW.

**Example Route Table Entry for Public Access:**

| Destination | Target     |
|-------------|------------|
| 0.0.0.0/0   | igw-xxxxxx |

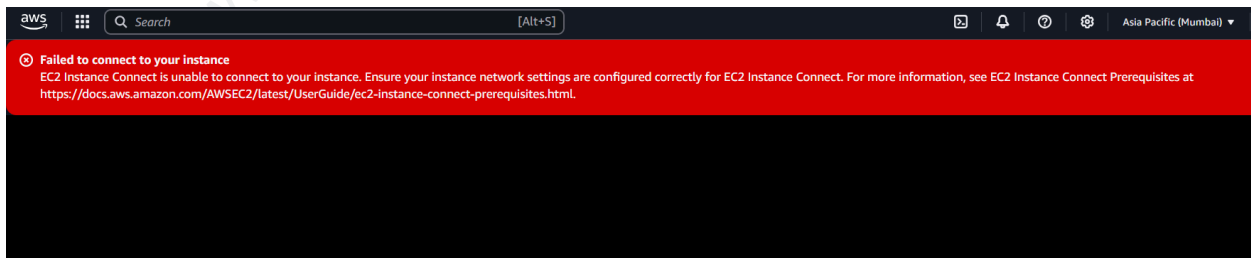
- This route allows **all outbound traffic** to go through the internet gateway.

## Step-by-Step Scenario: EC2 in Public Subnet (DemoVPC)

### Step 3: Launch EC2 in Public Subnet of DemoVPC

- **VPC:** DemoVPC
- **Subnet:** PublicSubnet a
- EC2 Instance launched with:
  - Public IP auto-assigned: ✓
  - Key pair created: ✓
  - Security group allowing SSH (port 22): ✓
- **Problem:** Cannot connect to EC2 via SSH ✗

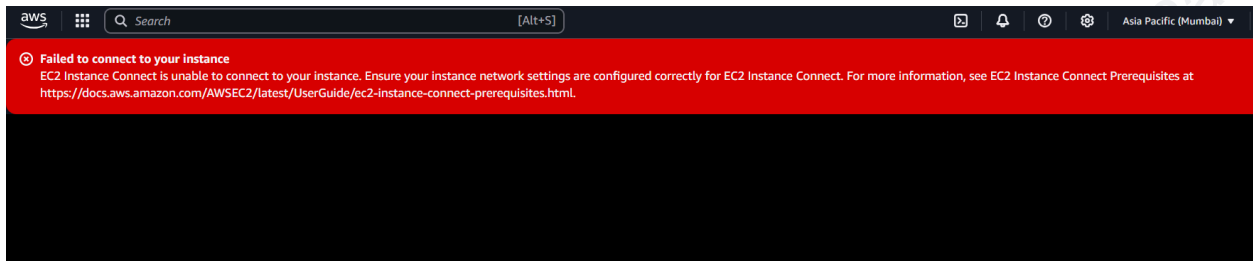
### Root Cause: No Internet Gateway (IGW) configured yet.



## Step 4: Create and Attach Internet Gateway

- Create an **Internet Gateway**, name it: **DemoIGW**.
- Attach **DemoIGW** to **DemoVPC**.
- **Problem persists:** Still cannot connect to EC2 ❌

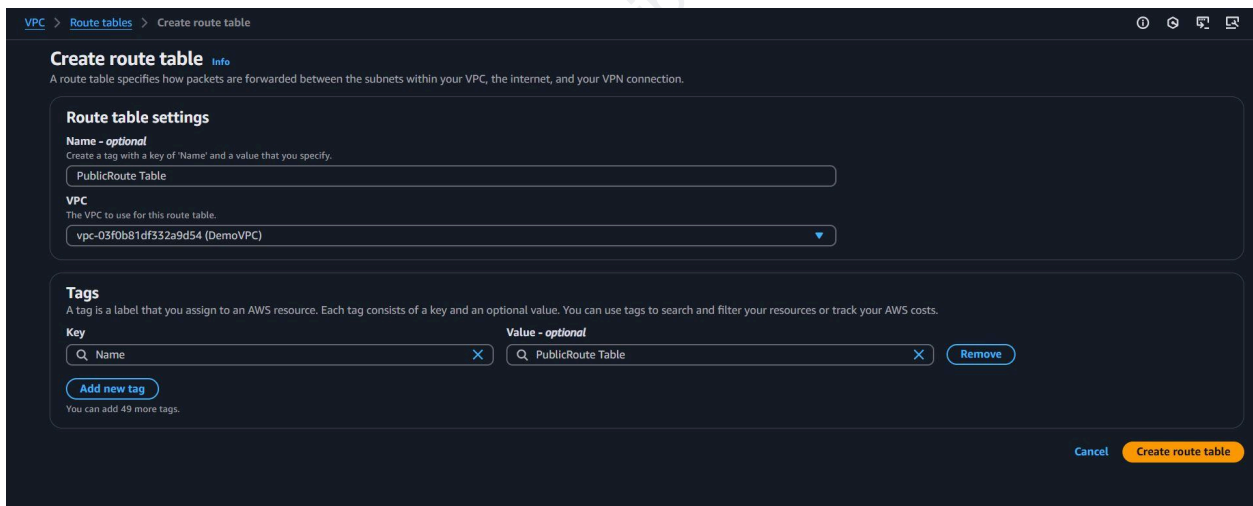
**Root Cause: No route in the route table directing traffic to the IGW.**



## Step 5: Create Two Route Tables

### ✓ Public Route Table

- Name: **PublicRoute Table**



## ✓ Private Route Table

- Name: PrivateRoute Table

**Create route table** Info

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

**Route table settings**

**Name - optional**  
Create a tag with a key of 'Name' and a value that you specify.  
PrivateRoute Table

**VPC**  
The VPC to use for this route table.  
vpc-03f0b81df332a9d54 (DemoVPC)

**Tags**  
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

**Key**  
Q Name X

**Value - optional**  
Q PrivateRoute Table X Remove

Add new tag  
You can add 49 more tags.

Cancel Create route table

**Route tables (1/3)** Info

Find resources by attribute or tag

VPC: vpc-03f0b81df332a9d54 X Clear filters

|                                     | Name               | Route table ID        | Explicit subnet associ... | Edge associations | Main | VPC           | Owner ID     |
|-------------------------------------|--------------------|-----------------------|---------------------------|-------------------|------|---------------|--------------|
| <input type="checkbox"/>            | PublicRoute Table  | rtb-01a63bbc78a13ed21 | -                         | -                 | No   | vpc-03f0b8... | 463470956366 |
| <input type="checkbox"/>            | -                  | rtb-05bb7830e7a68b990 | -                         | -                 | Yes  | vpc-03f0b8... | 463470956366 |
| <input checked="" type="checkbox"/> | PrivateRoute Table | rtb-0a1c8e99ff63e2776 | -                         | -                 | No   | vpc-03f0b8... | 463470956366 |

**rtb-0a1c8e99ff63e2776 / PrivateRoute Table**

Details Routes Subnet associations Edge associations Route propagation Tags

**Routes (1)**

Filter routes

| Destination | Target | Status | Propagated |
|-------------|--------|--------|------------|
| 10.0.0.0/16 | local  | Active | No         |

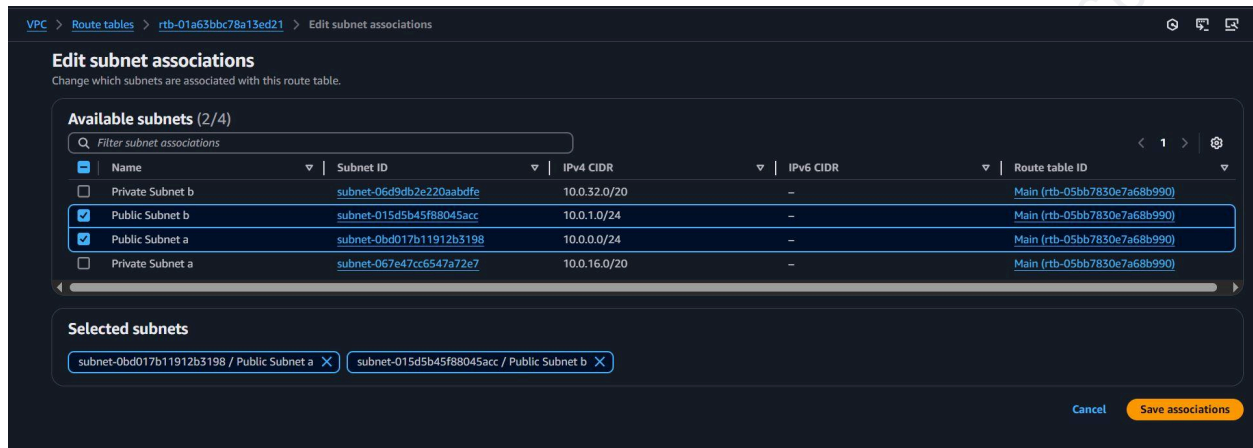
- Problem persists: Still cannot connect to EC2 ✗

**Root Cause:** We haven't done subnet association with the route table yet.

## Step 6: Associating Subnets to Route Tables

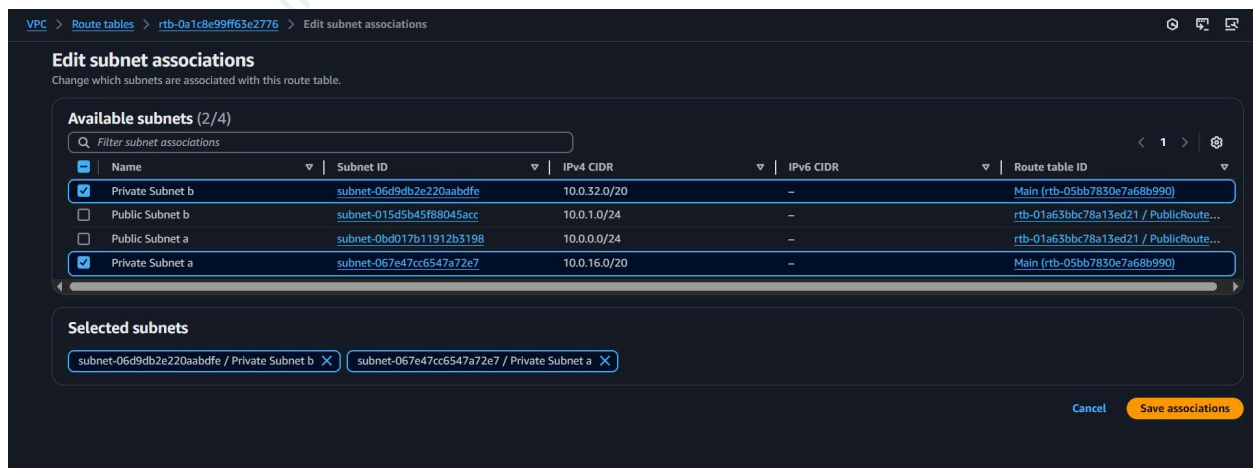
### ✓ Public Route Table

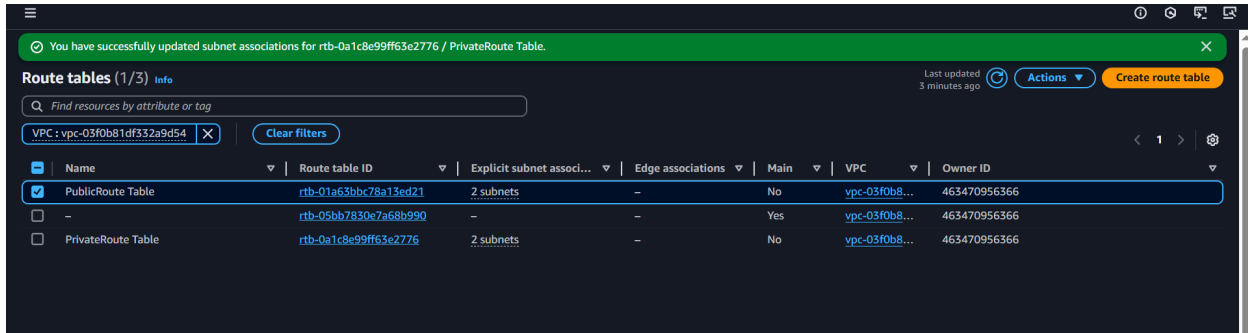
1. Navigate to **VPC Console** → **Route Tables**
2. Select the route table named **PublicRouteTable**
3. Click **“Edit subnet associations”**
4. Select the two public subnets (e.g., **PublicSubnet1**, **PublicSubnet2**)
5. Click **“Save associations”**



### ✓ Private Route Table

1. Go back to **Route Tables**
2. Select **PrivateRouteTable**  
Click **“Edit subnet associations”**
3. Select the two private subnets (e.g., **PrivateSubnet1**, **PrivateSubnet2**)
4. Click **“Save associations”**





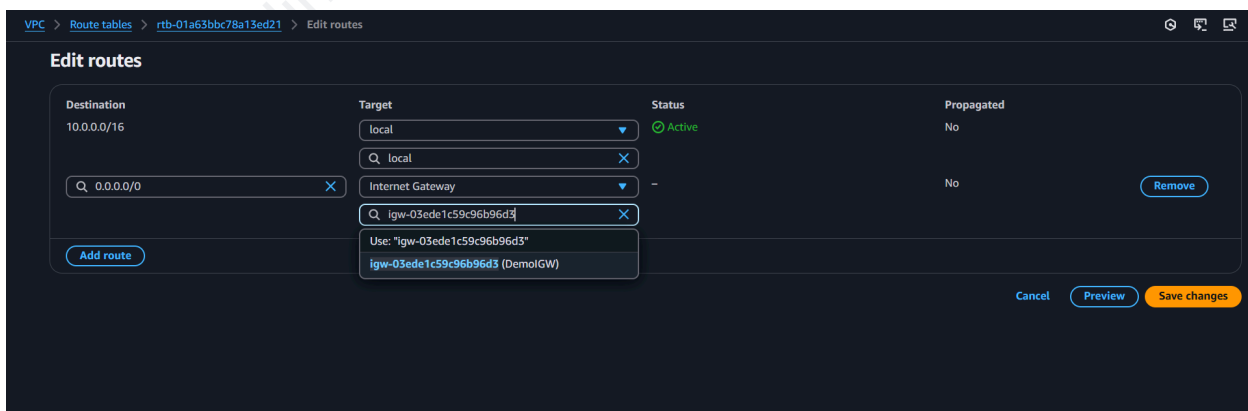
## Why Subnet Association Matters

- Without subnet association, **custom routes (like internet access)** won't apply.
- Associating subnets to the correct route tables ensures:
  - Public subnets route to **Internet Gateway**
  - Private subnets route **internally** or through **NAT Gateway**

## Step 7 :Edit Routes in Public Route Table

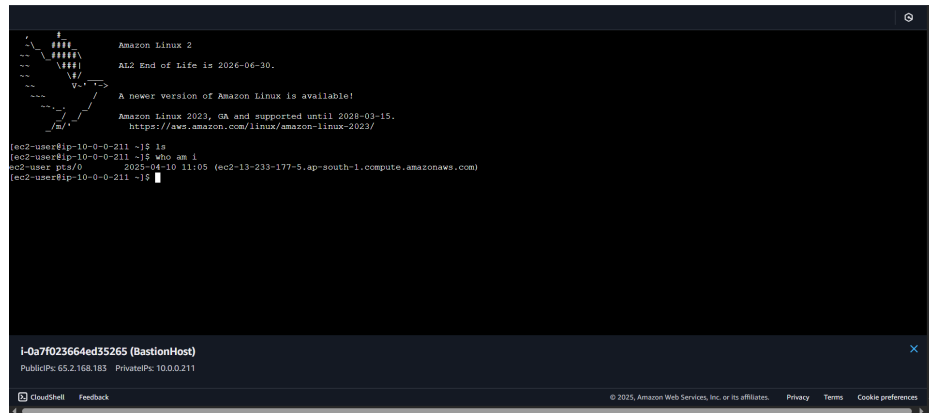
### Actions Performed:

1. Go to VPC Console → Route Tables
2. Select your PublicRouteTable
3. Click “Edit routes”
4. Click “Add route”
5. Set the following:
  - **Destination:** **0.0.0.0/0** (represents all IPv4 addresses)
  - **Target:** Select your Internet Gateway  
→ **igw-03ede1c59c96b96d3 (DemoIGW)** ✓



### 6. Click “Save changes”

Now you can see we can **connect to EC2 via SSH** ✓



```
Amazon Linux 2
AL2 End of Life is 2026-06-30.

A newer version of Amazon Linux is available!
Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-0-211 ~]$ ls
[ec2-user@ip-10-0-0-211 ~]$ whoami
ec2-user
[ec2-user@ip-10-0-0-211 ~]$
```

## Component

## Configured?

EC2 instance with public IP



Security Group allows SSH (22)



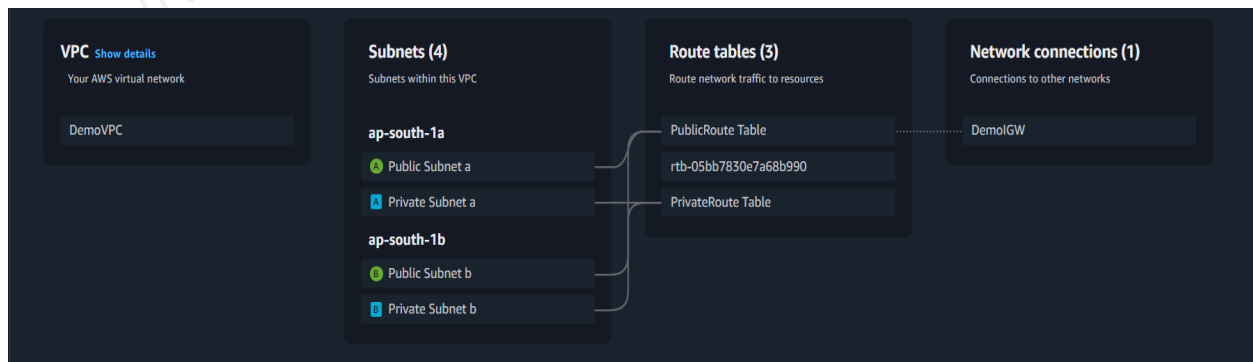
Internet Gateway attached to VPC



Route Table with 0.0.0.0/0 → IGW



Subnet associated with Route Table



## What is a Bastion Host ?

A **Bastion Host** (also known as a jump box) is a **special-purpose EC2 instance** used to securely connect to **private instances** (located in private subnets) in your VPC.

It acts as a **gateway** for administrative access (e.g., SSH or RDP) to resources in **private subnets**, which are **not directly accessible** from the internet.

## Why Use a Bastion Host?

### 1. Secure Access to Private Instances

- You **launch the bastion host in a public subnet** with a public IP.
- Then, **SSH/RDP into it from your local machine**, and from there, connect to private EC2s using their private IPs.

### 2. Minimize Exposure to Internet

- Only **one instance (the bastion)** is exposed to the internet.
- All other instances **remain in private subnets, isolated** from direct internet access.

### 3. Tight Security Control

- Bastion hosts can be locked down using:
  - **Security groups** (allow only your IP on port 22/3389)
  - **IAM policies**
  - **Session Manager (optional)**

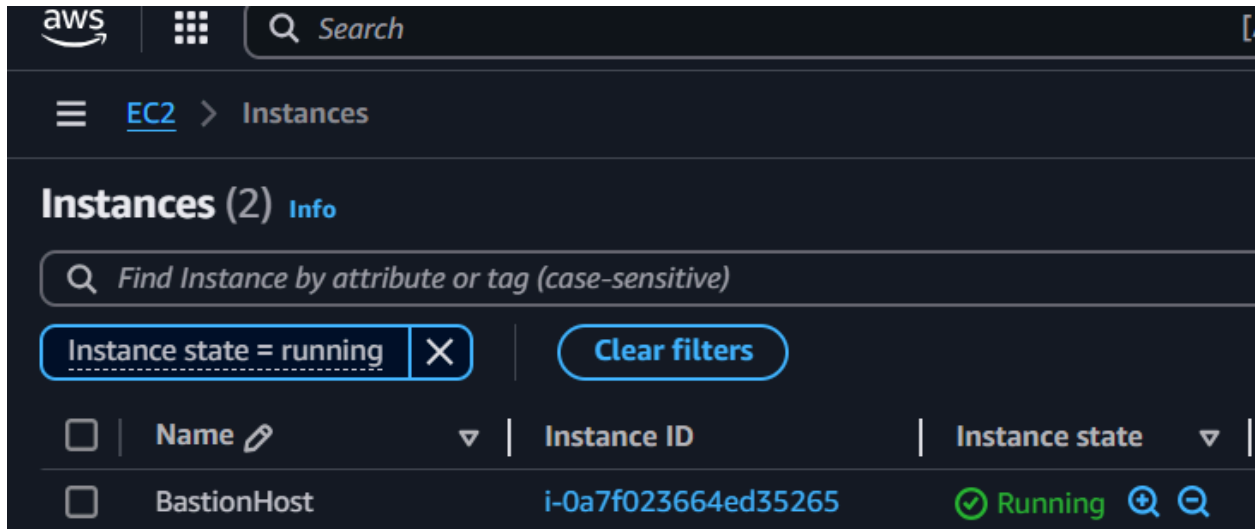
## Using Bastion Host to Access Private EC2 Instances

### Step-by-Step: Setup Access to Private EC2 Using Bastion Host

#### 1. Rename Public Instance to “BastionHost”

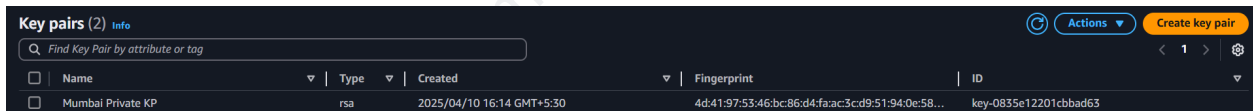
- Your existing EC2 instance in Public Subnet A is now renamed as:  
**BastionHost**





## ✓ 2. Create a New Key Pair

- Key Pair Name: Mumbai Private KP
- Type: RSA
- Used for authenticating with the private EC2 instance



## ✓ 3. Launch Private EC2 Instance

- Instance Name: **PrivateEC2**
- Subnet: Private Subnet A in DemoVPC
- Key Pair: Mumbai Private KP
- Security Group: Same as BastionHost (allows internal communication between the two instances)

## Section: Connecting to PrivateEC2 via Bastion Host

### Step-by-Step: Connect to PrivateEC2 from BastionHost

Now that the BastionHost is deployed in the public subnet, and PrivateEC2 is deployed in the private subnet, let's securely connect to the PrivateEC2 using the BastionHost.

#### Steps:

##### 1 Connect to BastionHost using SSH:

```
bash
```

```
ssh ec2-user@<BastionHost-Public-IP>
```

##### 2 On the BastionHost, create a PEM file:

```
bash
```

```
CopyEdit
```

```
nano MumbaiPrivateKP.pem
```

- Paste the content of "**Mumbai Private KP.pem**" into this file.
- Save and exit using:
  - **Ctrl + O, Enter** → to save
  - **Ctrl + X** → to exit

##### 3 Set correct permissions:ash

```
chmod 400 MumbaiPrivateKP.pem
```

##### 4 SSH into the PrivateEC2 instance:

```
bash
```

```
ssh -i MumbaiPrivateKP.pem ec2-user@10.0.25.82
```

✓ You are now connected to your PrivateEC2 instance from within BastionHost!

## 📌 Observation

Now, try running:

bash

CopyEdit

ping www.google.com

```
[ec2-user@ip-10-0-0-211 ~]$ nano MumbaiPrivateKP.pem
[ec2-user@ip-10-0-0-211 ~]$ ls
MumbaiPrivateKP.pem
[ec2-user@ip-10-0-0-211 ~]$ chmod 400 MumbaiPrivateKP.pem
[ec2-user@ip-10-0-0-211 ~]$ ls
MumbaiPrivateKP.pem
[ec2-user@ip-10-0-0-211 ~]$ ls -lstr
ls: cannot access -: No such file or directory
ls: cannot access lstr: No such file or directory
[ec2-user@ip-10-0-0-211 ~]$ ls -lstr
total 4
4 -r----- 1 ec2-user ec2-user 1722 Apr 10 11:36 MumbaiPrivateKP.pem
[ec2-user@ip-10-0-0-211 ~]$ ssh -i MumbaiPrivateKP.pem ec2-user@10.0.25.82

#
~\  #####      Amazon Linux 2
~~ \_#####\
~~  \###|      AL2 End of Life is 2026-06-30.
~~   \#/
~~    v~' '->
~~~
~~~. /
~~~ /m/ '

A newer version of Amazon Linux is available!

Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-25-82 ~]$ ping www.google.com
PING www.google.com (142.251.42.36) 56(84) bytes of data.

^C
--- www.google.com ping statistics ---
7 packets transmitted, 0 received, 100% packet loss, time 6140ms

i-0a7f023664ed35265 (BastionHost)
PublicIPs: 65.2.168.183 PrivateIPs: 10.0.0.211

CloudShell Feedback
```

You will see that ping fails (no internet access).

This is because PrivateEC2 is in a private subnet, and no NAT Gateway has been configured yet.

## Why This Happens

- Instances in private subnets have no direct route to the internet.
- An Internet Gateway cannot be attached directly to private subnets.
- To allow outbound internet traffic (e.g., software updates, package installs), you need to use a NAT Gateway or NAT instance.

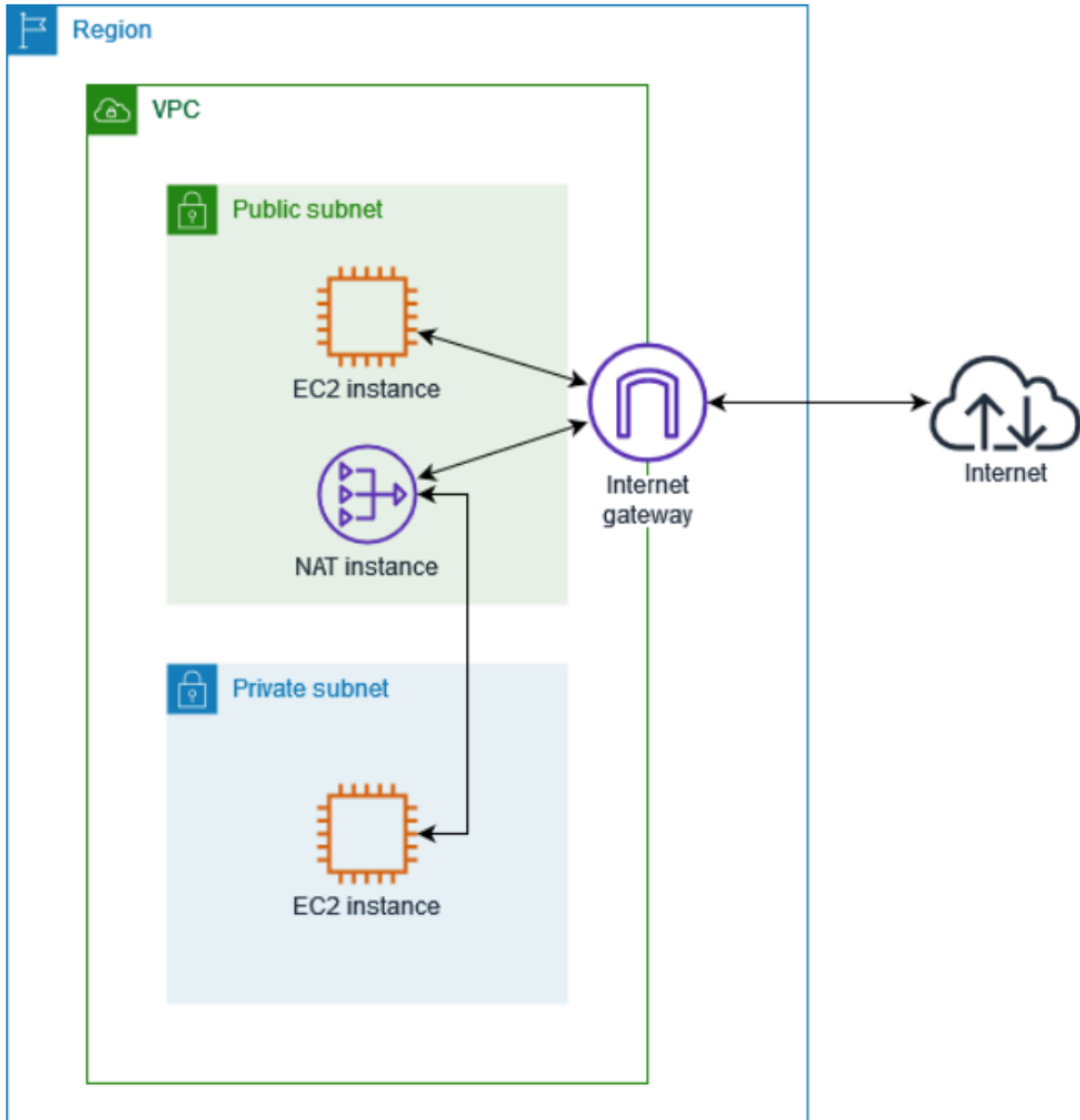
## What is a NAT (Network Address Translation)?

A NAT (specifically, a NAT instance or NAT gateway) enables instances in a private subnet to connect to the internet or other AWS services, but prevents the internet from initiating a connection with those instances.

---

### NAT Instance – Key Concepts:

- The route table associated with the private subnet sends internet-bound traffic from the private instances to the NAT instance, which must be located in a public subnet.
- The NAT instance then forwards the traffic to the internet gateway.
- The response from the internet returns to the NAT instance, which uses port mapping (high port numbers) to track requests and forwards the response to the correct private instance.
- Elastic IP: The NAT instance must have an Elastic IP (EIP) assigned to allow outbound internet access.



## How to Launch a NAT Instance:

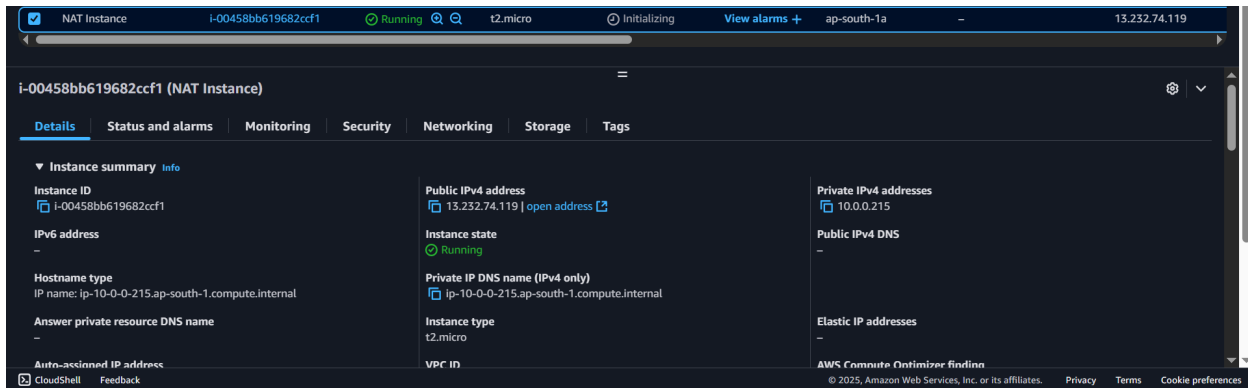
1. Go to EC2 Dashboard in the AWS Management Console.
2. Click on "Launch Instance."
3. Browse the AWS Marketplace.
4. Search for "NAT".
5. Select a NAT AMI (e.g., amzn-ami-vpc-nat for Amazon Linux).

  
Verified provider

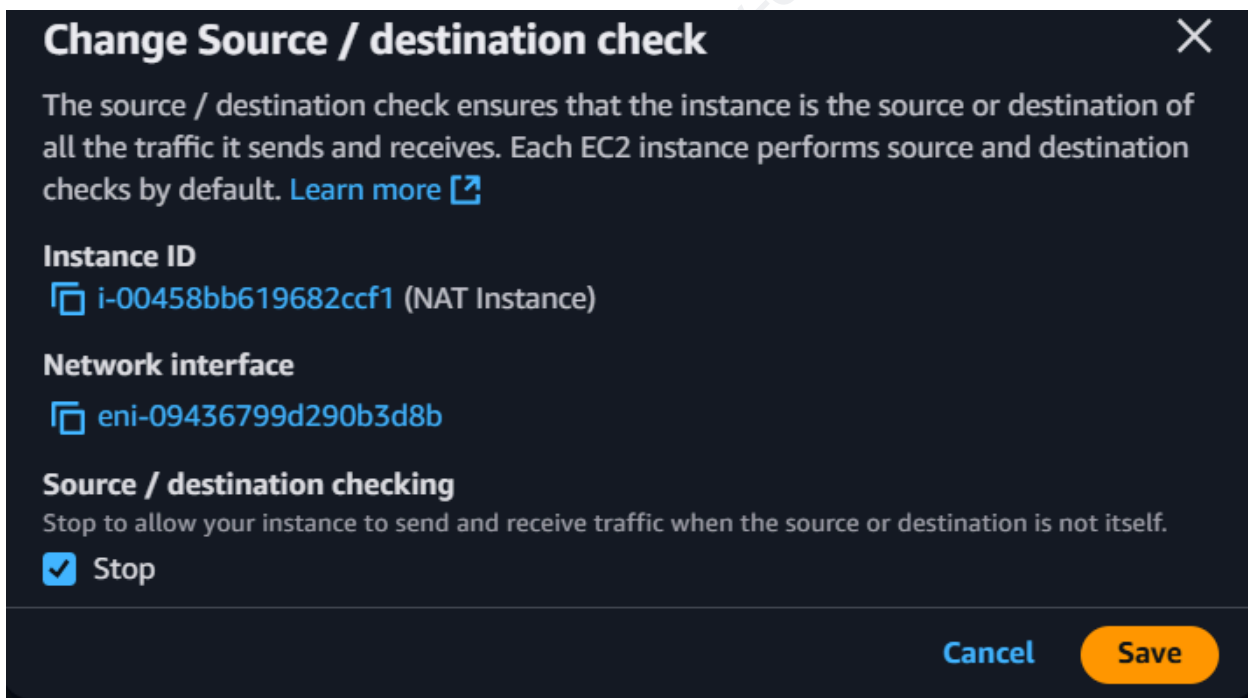
**amzn-ami-vpc-nat-2018.03.0.20230724.0-x86\_64-ebs**  
ami-022b58402848af996  
Amazon Linux AMI 2018.03.0.20230724.0 x86\_64 VPC HVM ebs  
OwnerAlias: amazon Platform: -- Architecture: x86\_64 Owner: 137112412989 Publish date: 2023-07-27 Root device type: ebs Virtualization: hvm ENA enabled: Yes

Select

6. Create a new Security Group (SG):
  - Allow inbound SSH (port 22) from your IP (for management).
  - Allow inbound HTTP/HTTPS or other protocols as needed.
  - Allow all outbound traffic.
7. Ensure the NAT instance is launched in a public subnet.



Stop Destination Cheking



## Connect to PrivateEc2 with BastionHost

```
Permission denied (publickey,gssapi-keyex,gssapi-mic).  
[ec2-user@ip-10-0-0-211 ~]$ ssh ec2-user@10.0.25.82 -i MumbaiPrivateKP.pem  
Last login: Thu Apr 10 11:37:21 2025 from 10.0.0.211
```

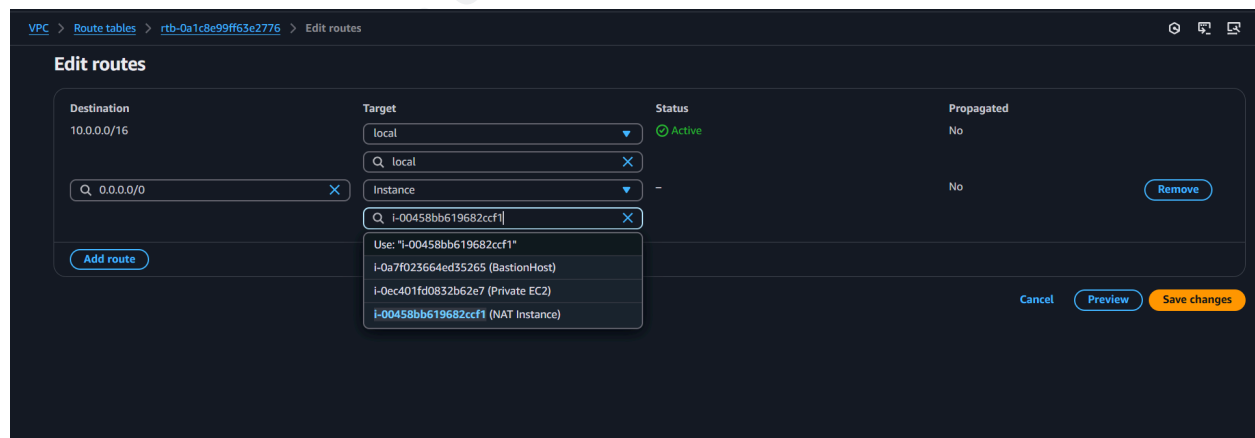
```
#_
~\####      Amazon Linux 2
~~\_#####\
~~   \###|    AL2 End of Life is 2026-06-30.
~~     \|
~~~~V~' '->

~~~~ /        A newer version of Amazon Linux is available!
~-.-./
_/m/'-/       Amazon Linux 2023, GA and supported until 2028-03-15.  
                https://aws.amazon.com/linux/amazon-linux-2023/  
  
[ec2-user@ip-10-0-25-82 ~]$ ping googlle.com  
PING googlle.com (104.21.83.75) 56(84) bytes of data:  
^C  
--- googlle.com ping statistics ---  
6 packets transmitted, 0 received, 100% packet loss, time 5112ms  
  
[ec2-user@ip-10-0-25-82 ~]$ □
```

## Editing Route Table to Allow Internet Access via NAT Instance

 **Goal:**

Update the **PrivateRoute Table** to send internet-bound traffic (0.0.0.0/0) to the **NAT Instance**.

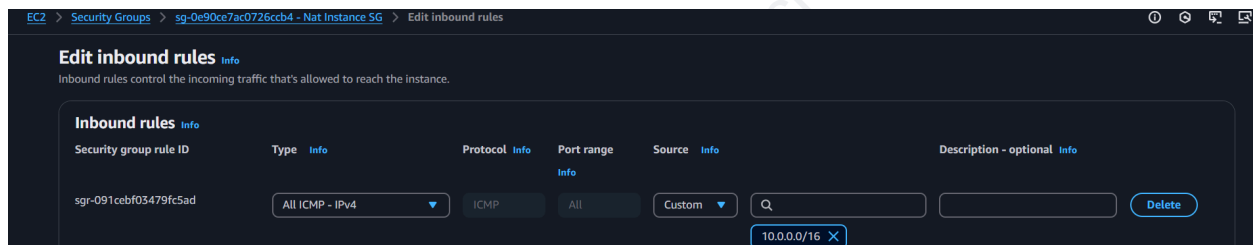


## Enabling ICMP Traffic to NAT Instance (Security Group Rule)

**Objective:** Allow network tools like **ping** (ICMP Echo Requests) to reach the NAT instance for diagnostic purposes.

### Steps:

1. Navigate to **EC2** → **Security Groups**
2. Find the **Security Group** attached to your NAT instance
3. Go to **Inbound Rules** → **Edit Inbound Rules**
4. Add new rule:
  - **Type:** **All ICMP - IPv4**
  - **Protocol:** **ICMP**
  - **Port Range:** **N/A**
  - **Source:** **Your internal CIDR block** (e.g., **10.0.0.0/16**) or **Anywhere** if testing
5. Click **“Save rules”**



## Test Internet Access from Private Subnet

1. Connect to an EC2 instance in the private subnet (via bastion host or SSM Session Manager).

Run the following command:

```
bash
ping google.com
```

Observe that:

- The ICMP echo reply is received.
- On the NAT instance, you can see incoming traffic, which confirms that the private instance is routing traffic through it to the internet.

 This verifies successful internet access from a private subnet using a NAT instance.



```
Amazon Linux 2
AL2 End of Life is 2026-06-30.

A newer version of Amazon Linux is available!
Amazon Linux 2023, GA and supported until 2028-03-15.
https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-25-82 ~]$ ping google.com
PING google.com (104.21.83.75) 56(84) bytes of data:
^C
--- google.com ping statistics ---
6 packets transmitted, 0 received, 100% packet loss, time 5112ms

[ec2-user@ip-10-0-25-82 ~]$ ls
[ec2-user@ip-10-0-25-82 ~]$ ping google.com
PING google.com (104.21.83.75) 56(84) bytes of data:
64 bytes from 104.21.83.75: icmp_seq=1 ttl=56 time=2.16 ms
64 bytes from 104.21.83.75: icmp_seq=2 ttl=56 time=1.76 ms
64 bytes from 104.21.83.75: icmp_seq=3 ttl=56 time=2.19 ms
64 bytes from 104.21.83.75: icmp_seq=4 ttl=56 time=2.62 ms
64 bytes from 104.21.83.75: icmp_seq=5 ttl=56 time=1.74 ms
64 bytes from 104.21.83.75: icmp_seq=6 ttl=56 time=1.90 ms
64 bytes from 104.21.83.75: icmp_seq=7 ttl=56 time=1.89 ms
64 bytes from 104.21.83.75: icmp_seq=8 ttl=56 time=2.33 ms
64 bytes from 104.21.83.75: icmp_seq=9 ttl=56 time=2.36 ms

i-0a7f023664ed35265 (BastionHost)
PublicPis: 65.2.168.183 PrivatePis: 10.0.0.211

CloudShell Feedback
```